

```

import 'dart:async';
import 'dart:math';
import 'package:flutter/material.dart';

void main() {
  runApp(const TicTacToeApp());
}

class TicTacToeApp extends StatelessWidget {
  const TicTacToeApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tic Tac Toe',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        scaffoldBackgroundColor:
          Colors.transparent, // transparent, bg handled by animated bg
        textTheme: const TextTheme(
          bodyMedium: TextStyle(fontFamily: 'Montserrat'),
        ),
      ),
      home: const SplashScreen(),
    );
  }
}

enum Player { X, O, none }

enum GameStatus { playing, xWins, oWins, draw }

enum GameMode { friend, bot }

enum BotDifficulty { easy, medium, hard }

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

```

```

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    Timer(const Duration(seconds: 2), () {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute<void>(builder: (_) => const ModeSelectionScreen()),
      );
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Stack(
        children: <Widget>[
          const AnimatedBackground(),
          Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: <Widget>[
                ShaderMask(
                  shaderCallback: (Rect bounds) {
                    return const LinearGradient(
                      colors: <Color>[Colors.blue, Colors.purple],
                      begin: Alignment.topLeft,
                      end: Alignment.bottomRight,
                    ).createShader(bounds);
                  },
                  child: const Text(
                    'Let\'s Play!',
                    style: TextStyle(
                      fontSize: 48,
                      fontWeight: FontWeight.bold,
                      color: Colors.white, // This color will be ignored due to ShaderMask
                    ),
                  ),
                const SizedBox(height: 20),
                const CircularProgressIndicator(
                  valueColor: AlwaysStoppedAnimation<Color>(Colors.white70),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}

```

```

    ),
  ),
],
),
);
}
}

```

```

class ModeSelectionScreen extends StatelessWidget {
  const ModeSelectionScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // We add the dynamic background here as well
      body: Stack(
        children: [
          const AnimatedBackground(),
          Scaffold(
            backgroundColor: Colors.transparent,
            appBar: AppBar(
              title: const Text('Tic Tac Toe'),
              centerTitle: true,
              elevation: 4,
              shadowColor: Colors.indigoAccent.withAlpha((255 * 0.3).round()),
            ),
            body: Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  ModeSelectionButton(
                    icon: Icons.group,
                    label: "Friend vs Friend",
                    color: Colors.indigo,
                    onTap: () => Navigator.push(
                      context,
                      MaterialPageRoute<void>(
                        builder: (_) => const FriendNameInputScreen(),
                      ),
                    ),
                  const SizedBox(height: 28),
                  ModeSelectionButton(
                    icon: Icons.smart_toy,

```

```

        label: "Bot vs User",
        color: Colors.deepOrange,
        onTap: () => Navigator.push(
            context,
            MaterialPageRoute<void>(
                builder: (_) => const BotDifficultyScreen(),
            ),
        ),
    ),
],
),
),
],
),
);
}
}

```

```

class ModeSelectionButton extends StatelessWidget {
  final IconData icon;
  final String label;
  final Color color;
  final VoidCallback onTap;

  const ModeSelectionButton({
    super.key,
    required this.icon,
    required this.label,
    required this.color,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton.icon(
      style: ElevatedButton.styleFrom(
        backgroundColor: color,
        padding: const EdgeInsets.symmetric(horizontal: 40, vertical: 18),
        elevation: 6,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
        shadowColor: color.withAlpha((255 * 0.6).round()),
      ),
      onPressed: onTap,
    );
  }
}

```

```

        icon: Icon(icon, size: 30),
        label: Text(
          label,
          style: const TextStyle(fontSize: 22, fontWeight: FontWeight.w600),
        ),
      );
    }
  }
}

```

```

class FriendNameInputScreen extends StatefulWidget {
  const FriendNameInputScreen({super.key});

```

```

  @override
  State<FriendNameInputScreen> createState() => _FriendNameInputScreenState();
}

```

```

class _FriendNameInputScreenState extends State<FriendNameInputScreen> {
  late final TextEditingController _playerXController;
  late final TextEditingController _playerOController;

```

```

  @override
  void initState() {
    super.initState();
    _playerXController = TextEditingController(text: 'Player X');
    _playerOController = TextEditingController(text: 'Player O');
  }

```

```

  @override
  void dispose() {
    _playerXController.dispose();
    _playerOController.dispose();
    super.dispose();
  }

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Stack(
        children: [
          const AnimatedBackground(),
          Scaffold(
            backgroundColor: Colors.transparent,
            appBar: AppBar(
              title: const Text("Enter Player Names"),

```

```

centerTitle: true,
elevation: 4,
shadowColor: Colors.indigoAccent.withAlpha((255 * 0.3).round()),
),
body: Center(
  child: Padding(
    padding: const EdgeInsets.all(32.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        TextField(
          controller: _playerXController,
          decoration: InputDecoration(
            labelText: 'Player X Name',
            hintText: 'Enter name for Player X',
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
            filled: true,
            fillColor: Colors.white.withAlpha((255 * 0.8).round()),
            prefixIcon:
              const Icon(Icons.close_rounded, color: Colors.indigo),
          ),
          style: const TextStyle(color: Colors.black87),
        ),
        const SizedBox(height: 24),
        TextField(
          controller: _playerOController,
          decoration: InputDecoration(
            labelText: 'Player O Name',
            hintText: 'Enter name for Player O',
            border: OutlineInputBorder(
              borderRadius: BorderRadius.circular(12),
            ),
            filled: true,
            fillColor: Colors.white.withAlpha((255 * 0.8).round()),
            prefixIcon: const Icon(Icons.circle_outlined,
              color: Colors.deepOrange),
          ),
          style: const TextStyle(color: Colors.black87),
        ),
        const SizedBox(height: 40),
        ElevatedButton.icon(
          icon: const Icon(Icons.play_arrow_rounded, size: 28),

```

```

label: const Text(
  "Start Game",
  style:
    TextStyle(fontSize: 20, fontWeight: FontWeight.w600),
),
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.green.shade600,
  padding: const EdgeInsets.symmetric(
    horizontal: 40, vertical: 16),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(16)),
  elevation: 6,
  shadowColor:
    Colors.greenAccent.withAlpha((255 * 0.6).round()),
),
onPressed: () {
  final String playerXName =
    _playerXController.text.trim().isEmpty
      ? _playerXController.text.trim()
      : 'Player X';
  final String playerOName =
    _playerOController.text.trim().isEmpty
      ? _playerOController.text.trim()
      : 'Player O';

  Navigator.pushReplacement(
    context,
    MaterialPageRoute<void>(
      builder: (_) => GameScreen(
        mode: GameMode.friend,
        playerXName: playerXName,
        playerOName: playerOName,
      ),
    ),
  );
},
),
],
),
),
),
),
],
),
),

```

```
class BotDifficultyScreen extends StatelessWidget {
  const BotDifficultyScreen({super.key});
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Stack(
      children: [
        const AnimatedBackground(),
        Scaffold(
          backgroundColor: Colors.transparent,
          appBar: AppBar(
            title: const Text("Select Bot Difficulty"),
            centerTitle: true,
            elevation: 4,
            shadowColor:
              Colors.deepOrangeAccent.withAlpha((255 * 0.3).round()),
          ),
          body: Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                DifficultySelectionButton(
                  label: "Easy",
                  difficulty: BotDifficulty.easy,
                  color: Colors.green.shade600,
                  onTap: () {
                    Navigator.push(
                      context,
                      MaterialPageRoute<void>(
                        builder: (_) => const GameScreen(
                          mode: GameMode.bot, difficulty: BotDifficulty.easy),
                      ),
                    );
                  },
                ),
                const SizedBox(height: 18),
                DifficultySelectionButton(
                  label: "Medium",
                  difficulty: BotDifficulty.medium,
```



```

        color: Colors.amber.shade700,
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute<void>(
              builder: (_) => const GameScreen(
                mode: GameMode.bot,
                difficulty: BotDifficulty.medium),
            ),
          );
        },
      ),
      const SizedBox(height: 18),
      DifficultySelectionButton(
        label: "Hard",
        difficulty: BotDifficulty.hard,
        color: Colors.red.shade700,
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute<void>(
              builder: (_) => const GameScreen(
                mode: GameMode.bot, difficulty: BotDifficulty.hard),
            ),
          );
        },
      ),
    ],
  ),
),
],
),
);
}
}

```

```

class DifficultySelectionButton extends StatelessWidget {
  final String label;
  final BotDifficulty difficulty;
  final Color color;
  final VoidCallback onTap;

  const DifficultySelectionButton({

```

```
super.key,  
required this.label,  
required this.difficulty,  
required this.color,  
required this.onTap,  
});
```

```
@override  
Widget build(BuildContext context) {  
  return ElevatedButton(  
    style: ElevatedButton.styleFrom(  
      backgroundColor: color,  
      padding: const EdgeInsets.symmetric(horizontal: 40, vertical: 16),  
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(14)),  
      elevation: 6,  
      shadowColor: color.withAlpha((255 * 0.6).round()),  
    ),  
    onPressed: onTap,  
    child: Text(  
      label,  
      style: const TextStyle(fontSize: 20, fontWeight: FontWeight.w600),  
    ),  
  );  
}
```

```
class GameScreen extends StatefulWidget {  
  final GameMode mode;  
  final BotDifficulty? difficulty;  
  final String? playerXName;  
  final String? playerOName;  
  
  const GameScreen({  
    super.key,  
    required this.mode,  
    this.difficulty,  
    this.playerXName,  
    this.playerOName,  
  });  
  
  @override  
  State<GameScreen> createState() => _GameScreenState();  
}
```

```

class _GameScreenState extends State<GameScreen>
  with SingleTickerProviderStateMixin {
  List<List<Player>> board = List.generate(
    3,
    (_) => List.filled(3, Player.none),
  );
  Player currentPlayer = Player.X;
  GameStatus status = GameStatus.playing;

  int xWins = 0;
  int oWins = 0;
  int draws = 0;

  Timer? _turnTimer;
  int _timeLeft = 5;

  AnimationController? _animationController;

  late final String _actualPlayerXName;
  late final String _actualPlayerOName;

  @override
  void initState() {
    super.initState();
    _actualPlayerXName = widget.playerXName ?? 'Player X';
    _actualPlayerOName = widget.playerOName ?? 'Player O';

    if (widget.mode == GameMode.friend) {
      _startTurnTimer();
    }
    _animationController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 200),
      lowerBound: 1.0,
      upperBound: 1.1,
    );
  }

  @override
  void dispose() {
    _turnTimer?.cancel();
    _animationController?.dispose();
    super.dispose();
  }

```

```

void resetBoard() {
  setState(() {
    board = List.generate(3, (_) => List.filled(3, Player.none));
    currentPlayer = Player.X;
    status = GameStatus.playing;
    if (widget.mode == GameMode.friend) {
      _startTurnTimer();
    } else {
      _turnTimer?.cancel();
    }
  });
}

```

```

void resetScores() {
  setState(() {
    xWins = 0;
    oWins = 0;
    draws = 0;
    resetBoard();
  });
}

```

```

void makeMove(int row, int col) {
  if (board[row][col] != Player.none || status != GameStatus.playing) return;

  setState(() {
    board[row][col] = currentPlayer;
    _animationController?.forward(from: 0.0);

    if (_checkWin(currentPlayer)) {
      status = currentPlayer == Player.X ? GameStatus.xWins : GameStatus.oWins;
      if (currentPlayer == Player.X) {
        xWins++;
      } else {
        oWins++;
      }
      _turnTimer?.cancel();
      return;
    } else if (_checkDraw()) {
      status = GameStatus.draw;
      draws++;
      _turnTimer?.cancel();
      return;
    }
  });
}

```

```

    }

    currentPlayer = currentPlayer == Player.X ? Player.O : Player.X;

    if (widget.mode == GameMode.bot && currentPlayer == Player.O) {
        _turnTimer?.cancel();
        _botMove();
    } else {
        if (widget.mode == GameMode.friend) {
            _startTurnTimer();
        }
    }
    });
}

void _startTurnTimer() {
    _turnTimer?.cancel();
    _timeLeft = 5;

    _turnTimer = Timer.periodic(const Duration(seconds: 1), (Timer timer) {
        if (!mounted) return;
        setState(() {
            _timeLeft--;
            if (_timeLeft <= 0) {
                timer.cancel();
                _skipTurn();
            }
        });
    });
}

void _skipTurn() {
    if (status == GameStatus.playing && widget.mode == GameMode.friend) {
        currentPlayer = currentPlayer == Player.X ? Player.O : Player.X;
        _startTurnTimer();
    }
}

void _botMove() async {
    await Future<void>.delayed(const Duration(milliseconds: 400));

    if (widget.difficulty == BotDifficulty.easy) {
        _botEasy();
    } else if (widget.difficulty == BotDifficulty.medium) {

```

```

    if (!_botTryWin()) {
        _botEasy();
    }
} else {
    _botHard();
}
}

void _botEasy() {
    List<Point<int>> emptyCells = _getEmptyCells();
    if (emptyCells.isNotEmpty()) {
        final Point<int> move = emptyCells[Random().nextInt(emptyCells.length)];
        makeMove(move.x, move.y);
    }
}

bool _botTryWin() {
    for (Point<int> cell in _getEmptyCells()) {
        board[cell.x][cell.y] = Player.O;
        if (_checkWin(Player.O)) {
            makeMove(cell.x, cell.y);
            return true;
        }
        board[cell.x][cell.y] = Player.none;
    }
    return false;
}

void _botHard() {
    int bestScore = -9999;
    Point<int>? bestMove;

    for (Point<int> cell in _getEmptyCells()) {
        board[cell.x][cell.y] = Player.O;
        int score = _minimax(0, false);
        board[cell.x][cell.y] = Player.none;
        if (score > bestScore) {
            bestScore = score;
            bestMove = cell;
        }
    }
}

if (bestMove != null) {
    makeMove(bestMove.x, bestMove.y);
}

```

```
}  
}
```

```
int _minimax(int depth, bool isMaximizing) {  
    if (_checkWin(Player.O)) return 10 - depth;  
    if (_checkWin(Player.X)) return depth - 10;  
    if (_checkDraw()) return 0;
```

```
    if (isMaximizing) {  
        int bestScore = -9999;  
        for (Point<int> cell in _getEmptyCells()) {  
            board[cell.x][cell.y] = Player.O;  
            int score = _minimax(depth + 1, false);  
            board[cell.x][cell.y] = Player.none;  
            bestScore = max(score, bestScore);  
        }  
        return bestScore;  
    } else {  
        int bestScore = 9999;  
        for (Point<int> cell in _getEmptyCells()) {  
            board[cell.x][cell.y] = Player.X;  
            int score = _minimax(depth + 1, true);  
            board[cell.x][cell.y] = Player.none;  
            bestScore = min(score, bestScore);  
        }  
        return bestScore;  
    }  
}
```

```
List<Point<int>> _getEmptyCells() {  
    List<Point<int>> emptyCells = <Point<int>>[];  
    for (int r = 0; r < 3; r++) {  
        for (int c = 0; c < 3; c++) {  
            if (board[r][c] == Player.none) emptyCells.add(Point<int>(r, c));  
        }  
    }  
    return emptyCells;  
}
```

```
bool _checkWin(Player player) {  
    for (int i = 0; i < 3; i++) {  
        if (board[i][0] == player &&  
            board[i][1] == player &&  
            board[i][2] == player) {
```

```

        return true;
    }
    if (board[0][i] == player &&
        board[1][i] == player &&
        board[2][i] == player) {
        return true;
    }
}
if (board[0][0] == player &&
    board[1][1] == player &&
    board[2][2] == player) {
    return true;
}
if (board[0][2] == player &&
    board[1][1] == player &&
    board[2][0] == player) {
    return true;
}
return false;
}

```

```

bool _checkDraw() {
    for (List<Player> row in board) {
        for (Player cell in row) {
            if (cell == Player.none) return false;
        }
    }
    return true;
}

```

```

String _statusText() {
    switch (status) {
        case GameStatus.playing:
            return 'Current Player: ${currentPlayer == Player.X ? _actualPlayerXName :
_actualPlayerOName}';
        case GameStatus.xWins:
            return '🎉 $_actualPlayerXName Wins!';
        case GameStatus.oWins:
            return widget.mode == GameMode.bot
                ? '🤖 Bot Wins!'
                : '🎉 $_actualPlayerOName Wins!';
        case GameStatus.draw:
            return "It's a Draw!";
    }
}

```



```
}
```

```
Color _statusColor() {  
  switch (status) {  
    case GameStatus.playing:  
      return currentPlayer == Player.X  
        ? Colors.indigo.shade700  
        : Colors.deepOrange.shade700;  
    case GameStatus.xWins:  
      return Colors.indigo.shade700;  
    case GameStatus.oWins:  
      return Colors.deepOrange.shade700;  
    case GameStatus.draw:  
      return Colors.grey.shade700;  
  }  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Stack(  
    children: [  
      const AnimatedBackground(), // The dynamic gradient background  
      Scaffold(  
        backgroundColor: Colors.transparent,  
        appBar: AppBar(  
          title: Text(  
            widget.mode == GameMode.friend ? 'Friend vs Friend' : 'Bot vs User',  
            style: const TextStyle(fontWeight: FontWeight.w600),  
          ),  
          centerTitle: true,  
          elevation: 4,  
          shadowColor: _statusColor().withAlpha((255 * 0.3).round()),  
          backgroundColor: _statusColor(),  
        ),  
        body: SingleChildScrollView(  
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),  
          child: Column(  
            children: [  
              Text(  
                _statusText(),  
                style: TextStyle(  
                  fontSize: 26,  
                  fontWeight: FontWeight.bold,  
                  color: _statusColor(),  

```

```

    ),
  ),
  if (widget.mode == GameMode.friend &&
      status == GameStatus.playing) ...<Widget>[
    const SizedBox(height: 12),
    Stack(
      alignment: Alignment.center,
      children: [
        SizedBox(
          width: 52,
          height: 52,
          child: CircularProgressIndicator(
            value: _timeLeft / 5,
            color: _statusColor(),
            strokeWidth: 6,
          ),
        ),
        Text(
          '$_timeLeft',
          style: TextStyle(
            fontSize: 22,
            fontWeight: FontWeight.bold,
            color: _statusColor(),
          ),
        ),
      ],
    ),
  ],
  const SizedBox(height: 20),
  TicTacToeBoard(
    board: board,
    currentPlayer: currentPlayer,
    mode: widget.mode,
    status: status,
    onCellTap: makeMove,
    animationController: _animationController!,
  ),
  const SizedBox(height: 24),
  ScoreDisplay(
    xWins: xWins,
    oWins: oWins,
    draws: draws,
    mode: widget.mode,
    playerXName: _actualPlayerXName,
  ),

```

```

        playerOName: _actualPlayerOName,
      ),
      const SizedBox(height: 28),
      ControlButtons(
        onResetBoard: resetBoard,
        onResetScores: resetScores,
      ),
    ],
  ),
),
],
);
}
}

```

```

class TicTacToeBoard extends StatelessWidget {
  final List<List<Player>> board;
  final Player currentPlayer;
  final GameMode mode;
  final GameStatus status;
  final void Function(int row, int col) onCellTap;
  final AnimationController animationController;

```

```

  const TicTacToeBoard({
    super.key,
    required this.board,
    required this.currentPlayer,
    required this.mode,
    required this.status,
    required this.onCellTap,
    required this.animationController,
  });

```

```

MaterialColor? _getPlayerColorPalette(Player p) {
  switch (p) {
    case Player.X:
      return Colors.indigo;
    case Player.O:
      return Colors.deepOrange;
    case Player.none:
      return null;
  }
}

```

```

Widget _playerIcon(Player p) {
  if (p == Player.X) {
    return const Icon(
      Icons.close_rounded,
      size: 48,
      color: Colors.white,
      shadows: <Shadow>[
        Shadow(
          blurRadius: 5,
          color: Colors.black54,
          offset: Offset(1, 1),
        ),
      ],
    );
  } else if (p == Player.O) {
    return const Icon(
      Icons.circle_outlined,
      size: 48,
      color: Colors.white,
      shadows: <Shadow>[
        Shadow(
          blurRadius: 5,
          color: Colors.black54,
          offset: Offset(1, 1),
        ),
      ],
    );
  }
  return const SizedBox.shrink();
}

```

@override

```

Widget build(BuildContext context) {
  return AspectRatio(
    aspectRatio: 1,
    child: Container(
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(22),
        boxShadow: <BoxShadow>[
          BoxShadow(
            color: Colors.grey.shade400,
            blurRadius: 12,
            offset: const Offset(2, 4),

```

```

    ),
  ],
  color: Colors.white.withAlpha((255 * 0.9).round()),
),
child: GridView.builder(
  padding: const EdgeInsets.all(12),
  physics: const NeverScrollableScrollPhysics(),
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 3,
    crossAxisSpacing: 14,
    mainAxisSpacing: 14,
  ),
  itemCount: 9,
  itemBuilder: (BuildContext context, int index) {
    int row = index ~/ 3;
    int col = index % 3;

    Player cellPlayer = board[row][col];
    MaterialColor? playerPalette = _getPlayerColorPalette(cellPlayer);

    return GestureDetector(
      onTap: () {
        if (mode == GameMode.friend || currentPlayer == Player.X) {
          onCellTap(row, col);
        }
      },
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 300),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(16),
          gradient: cellPlayer != Player.none
            ? LinearGradient(
                colors: <Color>[
                  playerPalette!.shade300,
                  playerPalette.shade600,
                ],
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
              )
            : const LinearGradient(
                colors: <Color>[Colors.grey, Colors.grey],
              ),
          boxShadow: <BoxShadow>[
            if (cellPlayer != Player.none)

```

```

        BoxShadow(
          color: playerPalette!.shade900
            .withAlpha((255 * 0.7).round()),
          blurRadius: 6,
          offset: const Offset(1, 3),
        ),
      ],
    ),
    child: Center(
      child: ScaleTransition(
        scale: animationController,
        child: _playerIcon(cellPlayer),
      ),
    ),
  ),
);
},
),
);
}
}
}

```

```

class ScoreDisplay extends StatelessWidget {
  final int xWins;
  final int oWins;
  final int draws;
  final GameMode mode;
  final String playerXName;
  final String playerOName;

```

```

  const ScoreDisplay({
    super.key,
    required this.xWins,
    required this.oWins,
    required this.draws,
    required this.mode,
    required this.playerXName,
    required this.playerOName,
  });

```

```

Widget _scoreCard(String label, int score, Color color) {
  return Card(
    elevation: 6,

```

```

shadowColor: color.withAlpha((255 * 0.4).round()),
shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
child: Container(
  width: 110,
  padding: const EdgeInsets.symmetric(vertical: 20),
  alignment: Alignment.center,
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: <Widget>[
      Text(
        label,
        textAlign: TextAlign.center,
        style: TextStyle(
          fontSize: 18, // Reduced font size to accommodate longer names
          color: color,
          fontWeight: FontWeight.w600,
        ),
      ),
      const SizedBox(height: 8),
      Text(
        score.toString(),
        style: TextStyle(
          fontSize: 36,
          color: color,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
);
}

```

```

@override
Widget build(BuildContext context) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: <Widget>[
      _scoreCard(playerXName, xWins, Colors.indigo.shade700),
      _scoreCard('Draws', draws, Colors.grey.shade700),
      _scoreCard(
        mode == GameMode.bot ? 'Bot' : playerOName,
        oWins,
        Colors.deepOrange.shade700,

```

```

    ),
  ],
);
}
}

```

```

class ControlButtons extends StatelessWidget {
  final VoidCallback onResetBoard;
  final VoidCallback onResetScores;

```

```

  const ControlButtons({
    super.key,
    required this.onResetBoard,
    required this.onResetScores,
  });

```

```

  @override

```

```

  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: <Widget>[
        ElevatedButton.icon(
          icon: const Icon(Icons.refresh_rounded),
          label: const Text('Restart'),
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.indigo.shade600,
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(16),
            ),
            elevation: 7,
            shadowColor: Colors.indigoAccent.withAlpha((255 * 0.6).round()),
          ),
          onPressed: onResetBoard,
        ),
        ElevatedButton.icon(
          icon: const Icon(Icons.delete_forever_rounded),
          label: const Text('Reset Scores'),
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.deepOrange.shade600,
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(16),
            ),

```



```

        elevation: 7,
        shadowColor:
            Colors.deepOrangeAccent.withAlpha((255 * 0.6).round()),
    ),
    onPressed: onResetScores,
),
],
);
}
}

// ===== Dynamic Animated Background Widget =====

class AnimatedBackground extends StatefulWidget {
  const AnimatedBackground({super.key});

  @override
  State<AnimatedBackground> createState() => _AnimatedBackgroundState();
}

class _AnimatedBackgroundState extends State<AnimatedBackground>
  with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<Color?> _color1;
  late Animation<Color?> _color2;
  late Animation<Color?> _color3;

  final List<Color> colors = <Color>[
    Colors.indigo.shade900,
    Colors.deepPurple.shade700,
    Colors.blue.shade600,
    Colors.cyan.shade700,
    Colors.deepOrange.shade600,
    Colors.pink.shade600,
  ];

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 10),
    )..repeat();
  }
}

```

```
    _color1 = ColorTween(begin: colors[0], end: colors[1]).animate(_controller);  
    _color2 = ColorTween(begin: colors[2], end: colors[3]).animate(_controller);  
    _color3 = ColorTween(begin: colors[4], end: colors[5]).animate(_controller);  
}
```

```
@override  
void dispose() {  
    _controller.dispose();  
    super.dispose();  
}
```

```
@override  
Widget build(BuildContext context) {  
    return AnimatedBuilder(  
        animation: _controller,  
        builder: (BuildContext _, Widget? __) {  
            return Container(  
                decoration: BoxDecoration(  
                    gradient: LinearGradient(  
                        begin: Alignment.topLeft,  
                        end: Alignment.bottomRight,  
                        colors: <Color>[  
                            _color1.value ?? colors[0],  
                            _color2.value ?? colors[2],  
                            _color3.value ?? colors[4],  
                        ],  
                    ),  
                ),  
            );  
        },  
    );  
}
```